# System Analysis & Design deliverables for JobPulse

## 1. Problem Statement & Objectives

Problem statement:

Job seekers often need to visit multiple recruitment platforms to find suitable opportunities. Job information is distributed across different websites, making it difficult to compare vacancies, identify in-demand skills, and understand job market trends.

JobPulse addresses this problem by collecting job vacancy information from multiple sources, integrating it into a centralized database, and providing search, filtering, and analytical capabilities.

Main objectives:

- Collect job vacancies from multiple recruitment platforms.
- Clean and standardize job information.
- Store structured data in a centralized database.
- Allow users to search and filter job opportunities.
- Analyze job market trends and in-demand skills.
- Provide interactive dashboards and reports.

## 2. Database Design & Data Modeling

| Table | Main attributes |
|---|---|
| Jobs | Job_ID, Title, Company_ID, Location_ID, Source_ID, Salary, Job_Type, Posted_Date |
| Companies | Company_ID, Company_Name, Industry |
| Locations | Location_ID, City, Country |
| Sources | Source_ID, Source_Name, Website |
| Skills | Skill_ID, Skill_Name |
| Job_Skills | Job_ID, Skill_ID |

## 3. Data Flow & System Behavior

| Diagram | What it demonstrates |
|---|---|
| Use Case | User searches, filters and views jobs; administrator manages collection |
| Sequence Diagram | Job collection and user search interactions |
| Activity Diagram | Workflow from collecting a job to displaying it |
| State Diagram | Job status: New, Active, Expired |
| Class Diagram | Job, Company, Skill, User and DataSource classes |

## 4. UI/UX Design




## 5. Deployment & Integration

Recommended technologies:

| Component | Technology |
|---|---|
| Database | SQL |
| Data processing | Pandas , dbt |
| Scraping | Selenium, Requests |
| Visualization | Power BI |
| Workflow automation | Apache Airflow |
| Version control | GitHub |

## 6. Additional Deliverables

Include:

- API documentation if you expose APIs.
- Unit testing for data cleaning functions.
- Integration testing for the ETL pipeline.
- Validation of missing and duplicate records.
- User acceptance testing for search and filtering.
- Deployment instructions and a GitHub README.